

Day 6 : Material Theme & Customization

1. Introduction to Flutter Theming & Customization

In Flutter, **Theming** is the process of defining the visual appearance of an application from a single location. Instead of manually assigning colors, fonts, and styles to every widget, Flutter provides the **ThemeData** class that allows developers to create a consistent design throughout the application.

Theming helps maintain a professional user interface, improves code reusability, reduces repetitive code, and makes it easier to switch between Light Mode and Dark Mode. Large applications rely heavily on theming because it ensures that all screens follow the same design language and branding guidelines.

2. What is ThemeData?

ThemeData is the core class responsible for storing all visual properties of a Flutter application.

It controls:

- Colors
- Fonts
- Text Styles
- Buttons
- App Bars
- Cards
- Input Fields
- Navigation Bars
- Dialogs

When a theme is defined inside MaterialApp, every widget automatically inherits those styles.

```
MaterialApp(  
  theme: ThemeData(),  
)
```

3. Why Do We Need Themes?

Without themes:

```
AppBar(  
  backgroundColor: Colors.blue,  
)
```

Every AppBar would need separate styling.

With themes:

```
theme: ThemeData(  
  appBarTheme: AppBarTheme(  
    backgroundColor: Colors.blue,  
  ),  
)
```

All AppBars automatically use the same style.

Advantages

- Consistent UI Design
- Easier Maintenance
- Faster Development
- Better User Experience
- Supports Branding
- Supports Light and Dark Modes

4. Understanding Color Scheme

A Color Scheme is a collection of colors used throughout the application.

Flutter's Material 3 uses ColorScheme instead of individual color properties.

Main Color Roles

Property	Purpose
Primary	Main brand color
Secondary	Accent color
Surface	Cards, sheets, dialogs
Background	Screen background
Error	Error messages
OnPrimary	Text displayed on primary color
OnSurface	Text displayed on surfaces

```
ColorScheme.fromSeed(  
  seedColor: Colors.blue,  
)
```

5. Primary Color

The primary color represents the application's identity.

Examples:

- Facebook → Blue -
- WhatsApp → Green
- YouTube → Red

```
colorScheme: ColorScheme.fromSeed(  
  seedColor: Colors.blue,  
)
```

6. Secondary Color

Secondary color is used to highlight important elements.

Examples:

- Floating Action Buttons
- Active Icons
- Accent Components

```
colorScheme: ColorScheme(  
  primary: Colors.blue,  
  secondary: Colors.orange,  
)
```

7. Typography

Typography controls how text appears.

It includes:

- Font Size
- Font Weight
- Font Style
- Letter Spacing
- Line Height

Flutter provides TextTheme

```
textTheme: TextTheme(  
  headlineMedium: TextStyle(  
    fontSize: 24,  
    fontWeight: FontWeight.bold,  
  ),  
)
```

8. AppBar Theme

AppBarTheme controls all AppBars in the application.

```
appBarTheme: AppBarTheme(  
  backgroundColor: Colors.blue,  
  foregroundColor: Colors.white,  
  centerTitle: true,  
)
```

Properties

- backgroundColor
- foregroundColor
- elevation
- centerTitle
- titleTextStyle

9 . Button Theme

Buttons can be customized globally.

```
elevatedButtonTheme: ElevatedButtonThemeData(  
  style: ElevatedButton.styleFrom(  
    padding: EdgeInsets.all(16),  
  ),  
)
```

10 . Light Theme / Dark Theme

Light Theme is used in bright environments.

Dark Theme is used in low-light environments.

```
ThemeData(  
  brightness: Brightness.light,  
)
```

```
ThemeData(  
  brightness: Brightness.dark,  
)
```

11 . Theme Mode

ThemeMode decides which theme to use.

```
ThemeMode.light  
ThemeMode.dark  
ThemeMode.system
```

12 . Dynamic Color

Material 3 supports dynamic colors.

Dynamic colors adapt according to device wallpaper and system settings.

Benefits:

- Personalized UI
- Better visual consistency
- Modern Android experience

13 . Surface Tint

Material 3 introduces Surface Tint.

Used on:

- Cards
- Sheets
- Dialogs

Purpose:

- Creates depth
- Improves visual hierarchy

Code:

```
import 'package:flutter/material.dart';

Run | Debug | Profile
void main() {
  runApp(MyApp());
}

class MyApp extends StatefulWidget {
  @override
  State<MyApp> createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
  bool isDark = false;

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,

      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue,
          brightness: Brightness.light,
        ), // ColorScheme.fromSeed
        appBarTheme: AppBarTheme(
          centerTitle: true,
        ), // AppBarTheme
      ), // ThemeData
```

```
      darkTheme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue,
          brightness: Brightness.dark,
        ), // ColorScheme.fromSeed
      ), // ThemeData

      themeMode: isDark
        ? ThemeMode.dark
        : ThemeMode.light,

      home: HomePage(
        onToggle: () {
          setState(() {
            isDark = !isDark;
          });
        },
      ), // HomePage
    ); // MaterialApp
  }
}
```

```
class HomePage extends StatelessWidget {
  final VoidCallback onToggle;

  const HomePage({required this.onToggle});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Day 6 Theming"),
      ), // AppBar
      body: Center(
        child: ElevatedButton(
          onPressed: onToggle,
          child: Text("Toggle Theme"),
        ), // ElevatedButton
      ), // Center
    ); // Scaffold
  }
}
```

